

0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

<http://pages.di.unipi.it/corradini/>

COMP-26: RUSTC #1 – 27/04/2026

Aspects of Rust Compilation

- Recap of Ownership and Borrowing in Rust
- Impact on compilation
 - Ownership and borrowing are not runtime mechanisms
 - they are static concept that have to be enforced by the compiler
- rustc targets the LLVM compiler framework
- Typical analysis performed on LLVM IR
- LLVM IR too much low-level
- Need of additional IR's in rustc
- rustc compilation pipe, with HIR, THIR, MIR

On Rust

- Development started in 2006 by **Graydon Hoare** at Mozilla.
- In 2010 shift from the initial compiler in OCaml to a self-hosting compiler written in Rust, **rustc**: it successfully compiled itself in 2011.
- **Bootstrapping** is used to generate new version of the compiler
- **rustc** uses **LLVM** as its back end.

On RUST goals and syntax

- **Rust** is a general purpose, system programming language with a focus on safety, especially safe concurrency, supporting both functional and imperative paradigms
- Main goal: ensuring safety without penalizing efficiency
- Concrete syntax similar to C and C++ (blocks, `if-else`, `while`, `for`), `match` for pattern matching
- Nearly every part of a function body is an expression (including `if-else`).
- No Runtime required (GC, Dynamic typing/binding,...)
- More control (over memory allocation/destruction...)

Rust overview

Performance, as with C

- Rust compilation to object code for bare-metal performance

But, supports memory safety:

- No null pointers
- No dangling pointers
- No double frees
- No data races
- No iterator invalidation
- Out-of-bound array accesses not allowed [→ C++ 26]

With low overhead

- Rules for memory safety checked mostly statically
- Zero-cost abstraction in managing memory (i.e. no garbage collection)

Via

- Advanced type system
- Ownership, borrowing, and lifetime concepts to prevent memory corruption issues [based on RAI]

But at a cost

- Cognitive cost to programmers who must think more about rules for using memory and references as they program

Resource Acquisition Is Initialization

- The Resource Acquisition Is Initialization (RAII) programming idiom states that Resource allocation is done during object initialization, by the constructor, while resource deallocation (release) is done during object destruction (specifically finalization), by the destructor.
- Popular in modern C++. Small objects better allocated on stack. Large resources are on the heap (or elsewhere) and are *owned* by an object on the stack. The object is then responsible for releasing the resource in its destructor.
- The object is bound to the scope (function, block) where it is declared; when the scope closes it is reclaimed, together with any owned resource.
- Each resource has a unique owner.

Ownership System

- Rust has an ownership system, which supports ~~RAII in a strict way~~
- Based on the concepts of ownership and borrowing
- Ownership can be summarized by three rules:

[O1] Every value is owned by a variable, identified by a name or path;

[O2] Each value has at most one owner at a time;

[O3] When the owner goes out-of-scope, the value is reclaimed / destroyed / 'dropped'.

Move semantics of assignment

- By default, an assignment between variables has a **move semantics**: the ownership is moved from the RHS to the LHS (by [O2])

```
fn main() {  
    let x = Box::new(3);  
    let _y = x; // underscore to avoid 'unused' warning  
    println!("x = {}", x); // error!  
}
```

- For primitive types and types implementing the **Copy trait**, assignment has a **copy semantics**
- [O2] is satisfied because a new value is created

```
fn main() {  
    let x = 3;  
    let _y = x;  
    println!("x = {:?}", x); // OK  
}
```

```
fn main() {  
    let x = Option::Some(3);  
    let _y = x;  
    println!("x = {:?}", x); // OK  
}
```


Move semantics of parameter passing

- The same with parameter passing and function return

```
fn foo<T>(z: T) -> T { // polymorphic identity function
    z
}

fn main(){
    let x = Box::new(3);
    let _y = foo(x);
    println!("x == {}", x); // error
}
```

```
fn main(){
    let x = 3;
    let _y = foo(x);
    println!("x == {}", x); // OK
}
```

- Any value passed to the function will be reclaimed when it returns, as the formal parameters gets out of scope
- Only the returned value can survive (tuples to return more)

```
fn main(){
    let mut x = Box::new(3);
    x = foo(x);
    println!("x == {}", x); // OK
}
```

Borrowing

- Ownership rules are too restrictive.
- A resource can be borrowed from its owner (via assignment or parameter passing).
- To guarantee memory safety, borrowing rules ensure that **ALIASING** and **MUTABILITY cannot coexist**
- Values can be passed
 - by immutable reference (with **`x = &y`**)
 - by mutable reference (with **`x = &mut y`**)
 - or by value (with **`x = y`**)

Borrowing Rules

[B1] At most one mutable reference to a resource can exist at any time

[B2] If there is a mutable reference, no immutable references can exist

[B3] If there is no mutable reference, several immutable references to the same resource can exist

- During borrowing, ownership is reduced or suspended:

[B4] Owner cannot free or mutate its resource while it is immutably borrowed

[B5] Owner cannot even read its resource while it is mutably borrowed

Borrowing: examples

- [B1] At most one mutable reference to a resource can exist at any time
- [B2] If there is a mutable reference, no immutable references can exist
- [B3] If there is no mutable reference, several immutable references to the same resource can exist

```
let mut s = String::from("example");  
let r1 = &mut s;  
let r2 = &mut s;  
println!("{}", r1, r2);    // does not compile by rule B1
```

```
let mut s = String::from("example");  
let r1 = &s;  
let r2 = &mut s;  
println!("{}", r1, r2);    // does not compile by rule B2
```

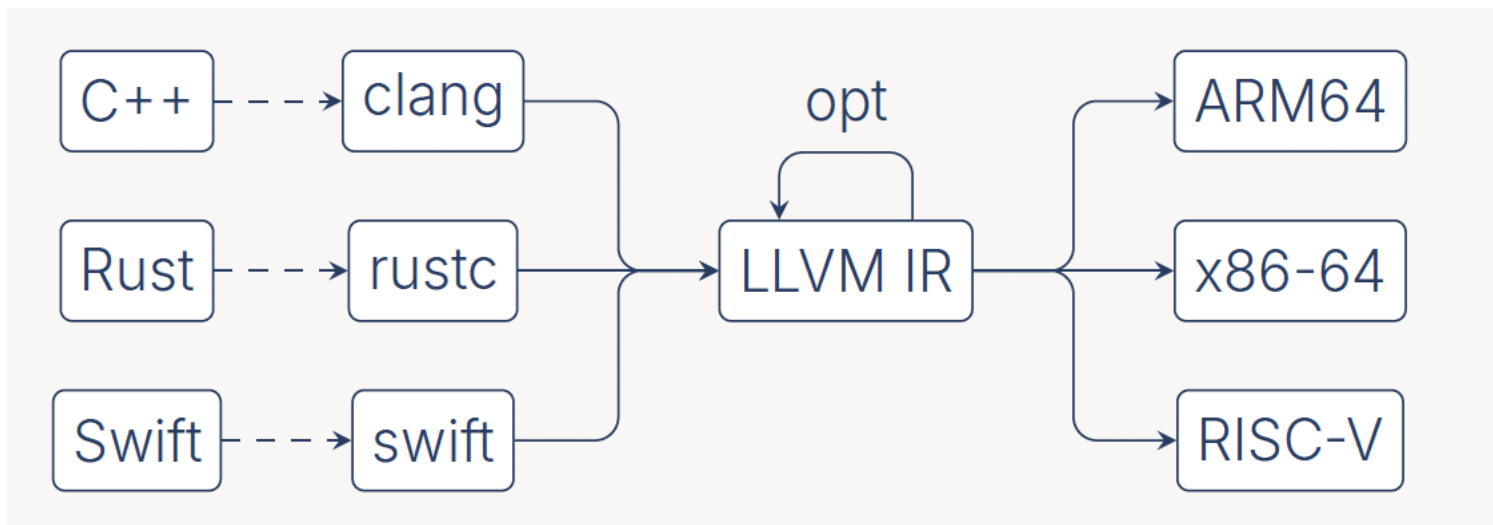
```
let s = String::from("example");  
let r1 = &s;  
let r2 = &s;  
println!("{}", r1, r2);    // ok by rule B3
```

Lifetimes

- A ***lifetime*** is a construct that the ***borrow checker*** uses to ensure the validity of the above rules
- Lifetimes are associated with each individual ownership and borrowing
- A lifetime begins when the ***ownership*** starts, and ends when it is ***moved*** / ***destroyed***.
- For borrowings, it ends ***where the borrowed value is accessed the last time***
- Lifetimes are mostly inferred. Sometimes must be made explicit using the same syntax of generics
- Using lifetimes, the compiler checks the validity of the rules of ownership and borrowing in the expected way
- In particular, it ensures that (the owner of) every borrowed variable/reference has a lifetime that is longer than the borrower [B4,B5]

Back to compiler: LLVM architecture

- Static analysis (e.g., type checking) in front end
 - Typically on the Abstract Syntax Tree
- Data-flow Analyses and optimizations on the LLVM IR
 - Typically on a Control Flow Graph

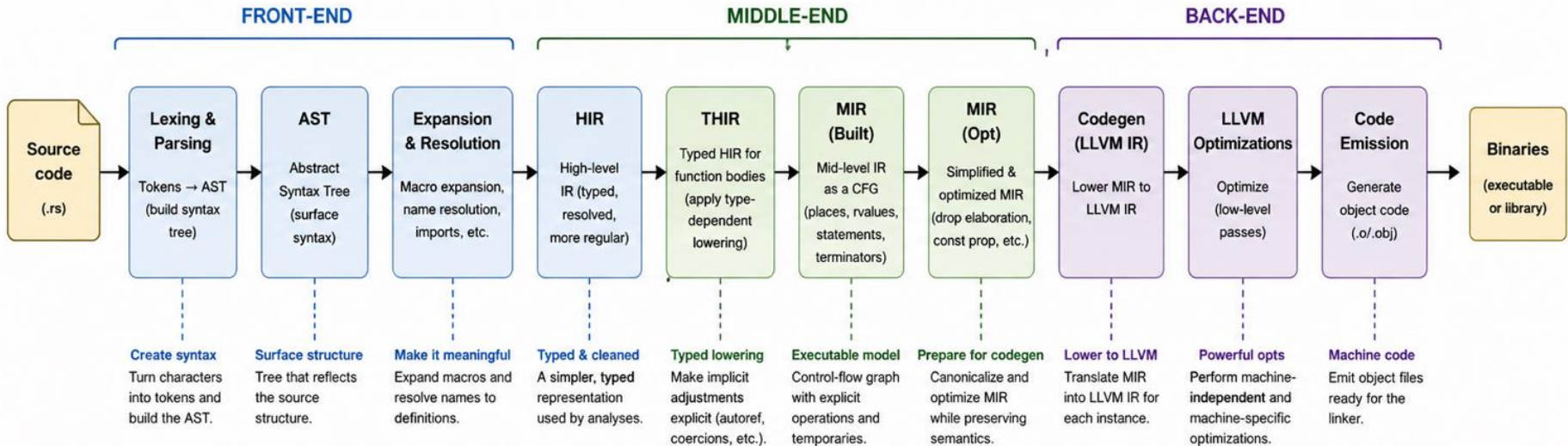


LLVM IR not suitable for borrow checker

- LLVM IR is **too low-level** for Rust's borrow checker
- Borrow checking is not just “alias analysis on machine-like code”. In Rust it is a **language-level, flow-sensitive safety analysis** over concepts such as:
 - places and projections (x, x.0, *p, v[i])
 - moves and partial moves
 - shared vs mutable borrows
 - region/lifetime constraints
 - drop semantics
 - desugared but still Rust-typed control flow
- Therefore **rustc** needs other IRs before generating the LLVM IR

```
fn push_len(v: &mut Vec<i32>) {  
    v.push(v.len() as i32);  
}
```

The rustc Compilation Pipeline



● FRONT-END

- Understand the program text.
- Resolve names and types.
- Produce HIR for the crate.

● MIDDLE-END

- Build and transform MIR.
- Run borrow checking and other analyses.
- Prepare MIR for codegen.

● BACK-END

- Monomorphize generic code.
- Generate LLVM IR.
- Optimize and emit object code.

● AFTER rustc

- Link object files and libraries.
- Produce final binaries.

Key idea

Ownership and borrowing are enforced statically in the middle-end over MIR, well before code reaches the backend.

AST = Abstract Syntax Tree • HIR = High-Level IR • THIR = Typed HIR • MIR = Mid-Level IR • CFG = Control-Flow Graph

Why is rustc interesting

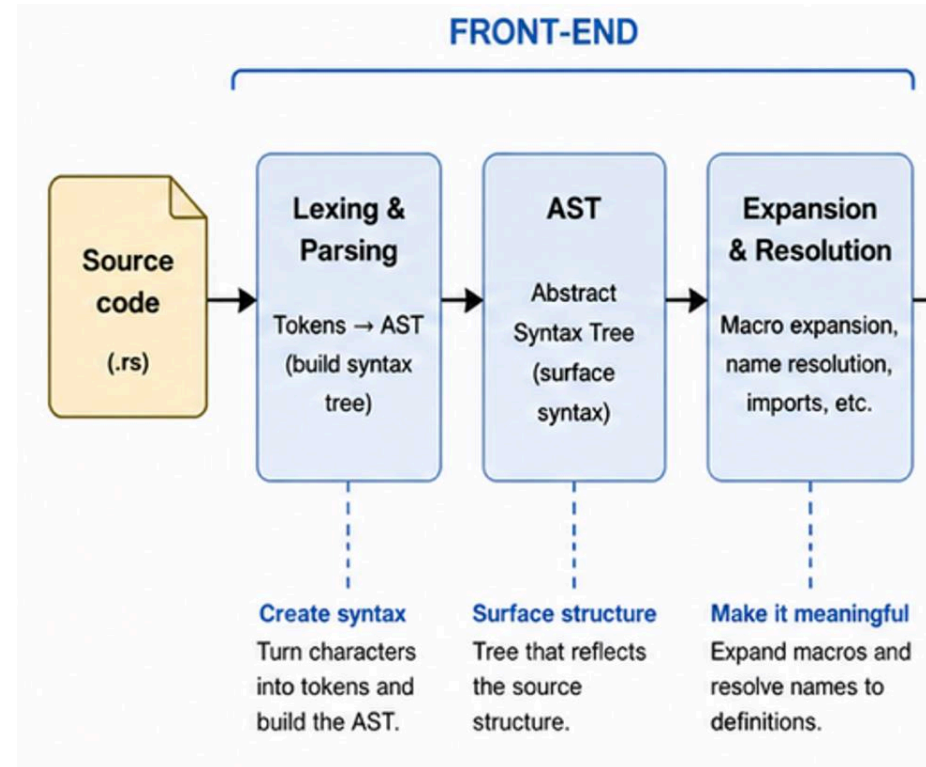
- Open source, open to contributions
- Classic topics such as CFGs, liveness, dataflow, constraints, and fixed points are visible as
language semantics
 - Not just in optimization or optional analysis phases
- The design as several unconventional
implementation choices (e.g. queries)

Queries in **rustc**

- **Queries** are the core abstraction used to structure the compiler as a *demand-driven system*.
- **rustc** is not run as a fixed sequence of passes: each piece of information (e.g., the type of a function, its MIR, or its borrow-checking result) is computed by a query that can depend on other queries.
- When the compiler needs some information, it invokes the corresponding query, which recursively ensures that all its dependencies are computed.
- Queries enable **incremental and cached compilation**.
- Query result are memoized, so if the same information is requested again, it can be reused without recomputation.
- When the source code changes, only the queries whose inputs have changed—and those depending on them—need to be recomputed.
- This makes compilation more efficient and scalable, especially for large projects, while also providing a clean modular structure for the compiler's implementation.

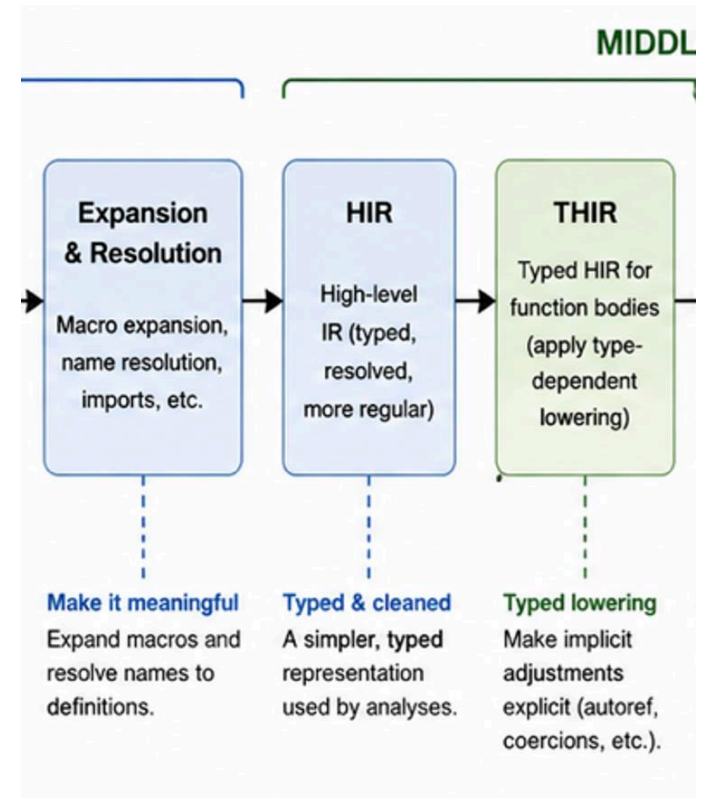
The Front-End

- Standard low-level lexer (source to tokens) [rustc-lexer](#)
- Higher-level lexer invoked by the parser
- Recursive descent parser [rustc-parse](#) generates the AST



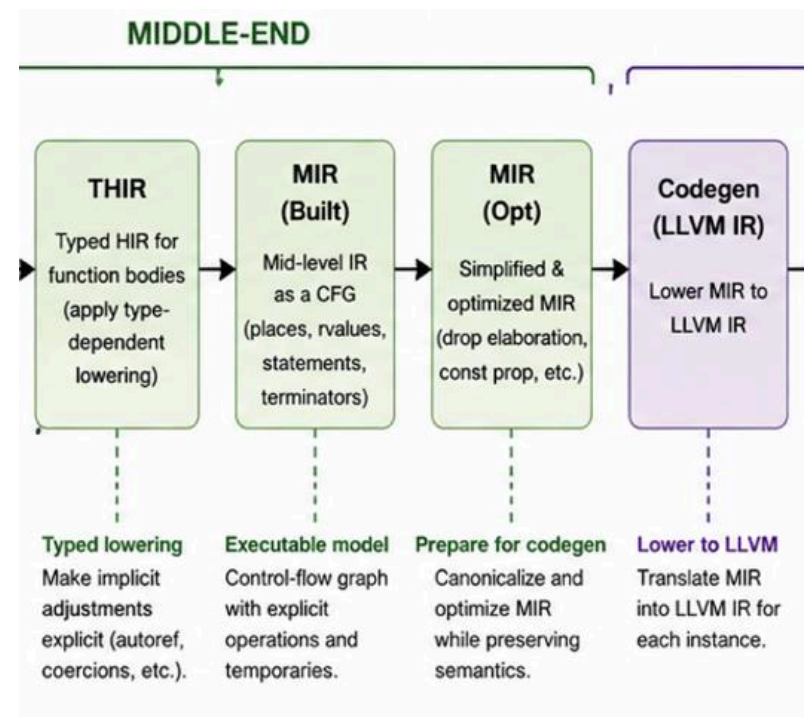
From Front- to Middle-End

- After macro expansion and name resolution, `rustc` lowers to HIR, a more regular syntax tree used by many front-end analyses
- Body-level typed information are added, obtaining the THIR



The Middle-End

- The THIR AST is translated into the MIR CFG, where control flow, temporaries, places, and drops become explicit.
- This is the IR on which the Borrow Checker is executed.
- The MIR is further optimized before the lowering to LLVM IR.
- The backend consumes verified and prepared MIR, creates concrete monomorphized instances, and delegates low-level optimization to LLVM.



Why an AST Alone Is Not Enough?

- **Too syntactic:** An AST mirrors user syntax: nested expressions, sugar, and source-level forms obscure order of evaluation and control-flow edges.
- **Repeated desugaring:** Without a central IR, every later pass would need to rediscover or reimplement the same desugarings and local semantics.
- **Hard CFG reasoning:** Borrow checking needs fine-grained points in the program and edges between them; those are awkward to express directly on an AST.
- **Executable shape:** MIR is closer to an executable model: blocks, statements, terminators, places, operands, and explicit temporaries.

Statement Translation: Source and AST

```
// Rust source inside a function
let z = if x > 0 { x + 1 } else { 0 };

// AST-like view
Stmt::Local {
  pat: Pat::Ident("z"),
  init: Expr::If {
    cond: Expr::Binary(Gt, Path("x"), Lit(0)),
    then: Block(Expr::Binary(Add, Path("x"), Lit(1))),
    else_: Block(Expr::Lit(0))
  }
}
```

- The Rust statement binds `z` to the value produced by an `if` expression.
- The AST is tree-shaped and preserves the nested expression structure.
- The block result is implicit: the last expression in each branch becomes the initializer.
- Later phases must expose both types and control flow more directly.

HIR: High-Level Intermediate Representation

- **Post expansion**: HIR is built after parsing, macro expansion, and name resolution, so many names and syntactic forms are already resolved.
- **Still readable**: It remains close to the programmer's program, which makes it suitable for front-end checks and diagnostics.
- **Some sugar removed**: Surface constructs can already be normalized; for example, some loop-oriented sugar is represented in more regular forms.
- **Not the borrow-check IR**: HIR is useful, but it is still too high-level for the detailed pointwise reasoning needed by MIR borrow checking.

THIR: Typed HIR for Function Bodies

- **Body-local IR**: THIR is produced for bodies and is intentionally temporary; `rustc` does not keep every THIR body globally forever.
- **Fully typed**: It makes type-dependent choices explicit, including adjustments such as autoref, autoderef, coercions, and overloaded calls.
- **Lowering bridge**: The MIR builder consumes THIR rather than raw HIR, because THIR already exposes important semantic decisions.
- **Conceptual role**: THIR is the bridge where one can see the shift from surface syntax toward operational facts needed by CFG construction.

Statement Translation: HIR and THIR

```
// let z = if x > 0 { x + 1 } else { 0 };  
// HIR-like view  
let z: i32 = if (x > 0) { x + 1 } else { 0 };  
  
// THIR-like view, typed and normalized  
LocalDecl z: i32  
Init = ExprKind::If {  
    cond: ExprKind::Binary(Gt, x:i32, 0_i32): bool,  
    then: ExprKind::Binary(Add, x:i32, 1_i32): i32,  
    else: 0_i32  
}  
// No CFG yet, but typed body facts are explicit.
```

- HIR is still close to syntax but has gone through important front-end lowering.
- The initializer and each branch are known to have type `i32`; the condition has type `bool`.
- THIR records adjustments and typed operator meanings before MIR construction.
- The MIR builder consumes the typed body and introduces temporaries and blocks.

MIR: Mid-Level IR as a Typed CFG

- **Basic blocks**: MIR organizes a function into basic blocks, each ending with a terminator such as return, goto, switch, call, or unwind.
- **Places and operands**: A place denotes storage such as a local, field, dereference, or indexed location; operands are copied, moved, or constant values.
- **Return place**: The function result is a distinguished local, usually printed as `_0`; arguments and temporaries are printed as `_1`, `_2`, and so on.
- **Central role**: MIR is used for safety analysis, transformations, optimization preparation, const evaluation, and eventual code generation.

MIR Dialects and Phases

- **MIR Built:** Initial MIR is constructed from THIR and still contains analysis-oriented constructs and source-level annotations.
- **MIR Analysis:** Borrow checking, NLL, move analysis, initialization checks, and dataflow reasoning operate on analysis MIR.
- **MIR Runtime:** After borrowck, cleanup and elaboration remove artifacts that were needed for checking but not for runtime execution.
- **Same structure, new contract:** The underlying MIR data structures are related across phases, but the semantic promises expected by each compiler stage change.

Statement Translation: MIR

```
//MIR-like of: let z = if x > 0 { x + 1 } else { 0 };  
let mut _z: i32;  
let mut _t0: bool;  
let mut _t1: i32;  
  
bb0: { _t0 = Gt(copy _x, const 0_i32);  
      switchInt(copy _t0) -> [0: bb2, otherwise: bb1]; }  
  
bb1: { _t1 = Add(copy _x, const 1_i32);  
      _z = move _t1;  
      goto -> bb3; }  
  
bb2: { _z = const 0_i32; goto -> bb3; }  
bb3: { /* z initialized here */ }
```

- The single expression becomes an explicit diamond control-flow graph.
- The condition and branch computation are stored in named temporaries
- Each branch writes to the same destination place, so initialization can be checked path-sensitively.
- This form is suitable for liveness, definite assignment, borrow checking, and lowering.